Day 1 – Environment & Project Setup

Goals: Install tooling, scaffold the Django project, configure environment variables, run the dev server.

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Create and activate virtual environment
uv venv
source .venv/bin/activate

# Add dependencies
uv add django djangorestframework python-decouple celery redis pytest-d

# Scaffold project and app
django-admin startproject gamekey_platform .
python manage.py startapp games
```

Create `.env` in the project root:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
```

Update `gamekey_platform/settings.py` to use `python-decouple`:

```
from decouple import config

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)

# Verify setup
python manage.py runserver
```

- Git init, add `.gitignore` (include `.env`)

🎓 Instructor Teaching Plan

⏱ Time Breakdown (90 min)

Segment	Duration	Activity
Bootcamp intro & big picture	15 min	Diagram: what we're building end-to-end. Show the finished flow: user buys key → key expires → Celery fires webhook → publisher receives it. Make students care about the outcome before writing a single line.
Python ecosystem warmup	10 min	Briefly contrast `pip + venv` with `uv`. Why `uv`? Speed, lockfiles, reproducibility. Don't over-explain — run it live.

Live setup (students follow along)	25 min	Walk through every command in the Day 1 code block. Pause after `uv venv` and `source .venv/bin/activate` — confirm every student sees `(.venv)` in their prompt before continuing.
`python-decouple` and `.env`	15 min	Explain *why* secrets don't go in source code. Show what happens if you hard-code `SECRET_KEY` and push to GitHub. Set up `.env`, read it via `decouple`, add `.env` to `.gitignore`.
Git init & first commit	10 min	`git init`, stage, commit. Confirm `.env` is not tracked (`git status`).
Q&A + checkpoint verification	15 min	Every student must show the Django rocket page at `localhost:8000` before leaving.

📡 Key Concepts to Introduce

- **What is Django?** MTV framework (Model-Template-View). We'll use it purely as an API backend — no templates.
- **What is DRF?** A toolkit on top of Django for building REST APIs. Preview: it gives us serializers, viewsets, routers.
- **Virtual environments** — why isolation matters (dependency conflicts, reproducibility).
- **`DEBUG=True` vs `False`** — Django shows a detailed error page in debug mode. Always `False` in production.
- **Why `python-decouple`?** `os.environ.get()` works too, but `decouple` handles type casting, default values, and `.env` parsing in one clean API.

⚠️ Common Mistakes to Address

- **Forgetting to activate the venv** — the most common Day 1 error. Symptom: `django` not found even after `uv add`. Teach students to check `which python`.
- **Committing `.env` to git** — show *before* adding to `.gitignore` what `git status` reveals. Make the danger concrete.
- **`SECRET_KEY` still hard-coded** — students sometimes add `python-decouple` but forget to update `settings.py`. Verify by removing the `.env` line and confirming Django raises an error.
- **Windows path differences** — `.venv/Scripts/activate` not `.venv/bin/activate`. If any Windows users are present, flag this upfront.

❓ Check for Understanding

> "Why can't we just hard-code `SECRET_KEY = 'abc123'` in settings.py?"
>
> **Answer:** Hard-coding `SECRET_KEY` in source code poses a severe security risk. If the repository is pushed to a public platform (like GitHub), anyone can see the key. Attackers can use it to forge signed cookies, session data, or password reset tokens. Storing it in an environment variable (`.env` file) keeps it private and allows different values for development and production environments.
>
> "What would happen if two projects on the same machine installed different versions of `django` without virtual environments?"

Answer: Installing different versions globally would cause a conflict because Python can only resolve one version of a package in its global site-packages directory. Installing a different version for the second project would overwrite the version needed by the first project, breaking it. Virtual environments isolate dependencies per project, allowing each to run its required version independently.

"What does `DEBUG=True` change about how Django behaves?"

Answer: When `DEBUG=True`, Django displays detailed error traceback pages to the client (useful for debugging but a huge security leak in production because it exposes database queries, settings, and path variables). It also runs the built-in development server's auto-reloader and serves static/media files automatically. In production (`DEBUG=False`), it returns a generic 500 error page, requires `ALLOWED_HOSTS` to be set, and disables automated static file serving.

✓ Day Checkpoint

Student shows the Django welcome page at `http://127.0.0.1:8000/`. Their `.env` contains `SECRET_KEY` and `DEBUG`. Running `git log --oneline` shows at least one commit. `git status` shows `.env` as untracked (not staged).